

Final Neural Network Model - ReadMe

Numerical Model Setup:

The neural networks are trained on data generated from numerical models of the internal combustion engine (ICE) and the drivetrain. Compared to previous versions, no fundamental changes were made to the setup of these base models. Only vehicle-related parameters, such as vehicle mass and air resistance, were adjusted. The ICE model remains based on the EA189 engine, as test bench measurement data is still not available. This decision is final and will not be changed.

An important extension compared to earlier model versions concerns the cooldown behavior of the aftertreatment system during engine-off phases. Previously, this behavior could not be represented because two separate numerical models were required for data generation. This limitation has now been addressed by adding stationary cooldown cycles to the training dataset. As a result, the validation range of the ICE neural network has changed and must be interpreted carefully.

The ICE model now has two distinct valid operating regimes. When the engine is switched off, both **engine speed and fuel mass must be exactly zero**. When the engine is running, the valid operating range starts at an engine speed of 900 rpm and extends up to 4000 rpm. Fuel mass in this regime ranges from 3 mg to 70 mg. The model is explicitly not valid in the intermediate range between 0 and 900 rpm or between 0 and 3 mg of fuel mass. Operating the model in this region may lead to unphysical results or unstable simulations.

It is assumed that ambient conditions remain constant over the duration of one driving cycle, which is defined as 1800 seconds. The exact valid ranges are shown in [Table 1](#).

Table 1: Valid ranges of the ICE model

Parameter	Range
Speed (n_{ICE})	Engine off: 0 rpm
	Engine on: 900 rpm – 4000 rpm
Fuelmass (m_{fuel})	Engine off: 0 mg
	Engine on: 3 mg – 70 mg
Ambient temperature (T_{amb})	250 K – 310 K
Ambient pressure (p_{amb})	0.8 bar – 1.1 bar

The drivetrain model follows the same working principle as in previous versions. Apart from updated vehicle properties, no structural changes were applied. The drivetrain neural network is valid over a wide operating range (Table 2).

Table 2: Valid ranges of the Drivetrain model

Parameter	Range
SoC	0 – 1
n_{ICE}	0 rpm – 4500 rpm
τ_{EM2}	-421 Nm – 421 Nm
τ_{ICE}	-50 Nm – 300 Nm
Brake _{mech}	0 % – 100 %

Final Neural Network Model Structure:

The final neural network setup consists of three individual neural networks: an ICE network, a drivetrain network, and an additional drivetrain network referred to as Drivetrain Plus (Figure

1). The Drivetrain Plus network receives the same inputs as the standard drivetrain network but predicts additional internal drivetrain states. These outputs are not required for reinforcement learning applications but can be used to gain further insight into the operating conditions of the electric machines and the planetary gear set. For this reason, the Drivetrain Plus network is optional.

Two parameters are special. The ICE neural network predicts engine torque, which is the transfer output, that serves as cascaded input to the drivetrain models. The second special parameter is ICE speed that serves as input for all three neural network models.

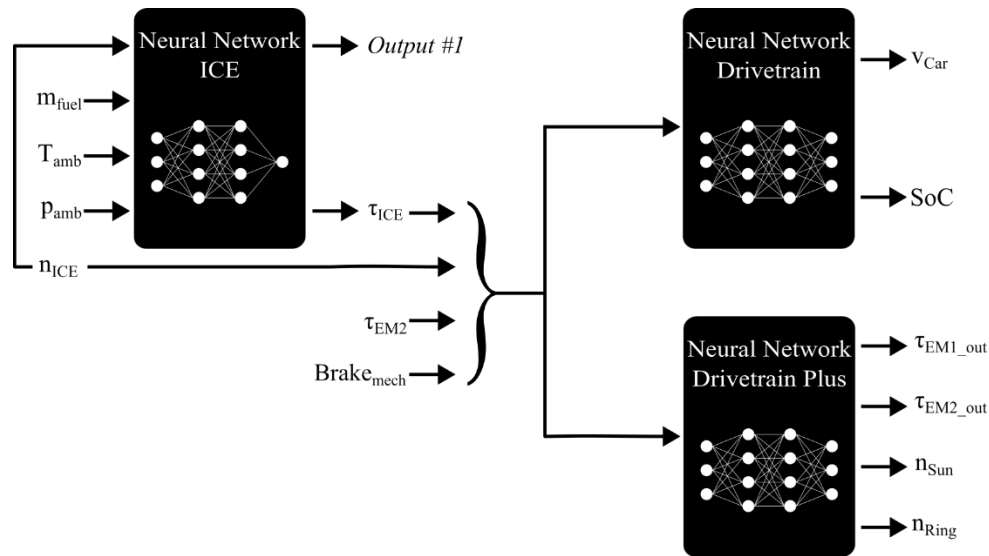


Figure 1: Final NN model setup

In the following all inputs and outputs of all of the three networks are displayed.

Table 3: In- and Outputs of neural network model ICE

Neural Network		Parameter	Description
ICE (EA189)	Inputs	n_{ICE} [rpm]	Control Input to all NN
		m_{fuel} [mg/cyl/cycle]	Control Input
		T_{amb} [K]	Boundary Input
		p_{amb} [bar]	Boundary Input
	Outputs (Output #1)	T_{ICE} [Nm]	Transfer Output
		m_{fuel_tot} [g/s]	Total burned Fuelmass
		NO_{x_eo} [g/s]	Raw emission
		CO_{eo} [g/s]	Raw emission
		THC_{eo} [g/s]	Raw emission
		T_{gas_eo} [K]	Gas temperature engine out (eo)
		NO_{x_tp} [g/s]	Tailpipe (tp) emission
		CO_{tp} [g/s]	Tailpipe emission
		CO_{2_tp} [g/s]	Tailpipe emission
		THC_{tp} [g/s]	Tailpipe emission
		T_{wall_SCR1} [K]	Wall temperature SCR1
		T_{wall_DOC} [K]	Wall temperature DOC
		T_{sub_DPF} [K]	Wall temperature DPF
		T_{wall_SCR2} [K]	Wall temperature SCR2
		T_{wall_SCR3} [K]	Wall temperature SCR3
		T_{gas_tp} [K]	Gas temperature Tailpipe

Table 4: In- and Outputs of neural network model Drivetrain

Neural Network		Parameter	Description
Drivetrain	Inputs	n_{ICE} [rpm]	Control Input to all NN
		T_{ICE} [Nm]	Transfer Input
		T_{EM2} [Nm]	Control Input
		$Brake_{mech}$ [%]	Control Input
	Output	v_{Car} [km/h]	Car Velocity
		SoC [-]	Battery State of Charge

Table 5: In- and Outputs of neural network model Drivetrain Plus

Neural Network		Parameter	Description
Drivetrain Plus	Inputs	n_{ICE} [rpm]	Control Input to all NN
		T_{ICE} [Nm]	Transfer Input
		T_{EM2} [Nm]	Control Input
		$Brake_{mech}$ [%]	Control Input
	Output	T_{EM1_out} [Nm]	Actual EM1 torque
		T_{EM2_out} [Nm]	Actual EM2 torque
		n_{Sun} [rpm]	Actual Sun Speed
		n_{Ring} [rpm]	Actual Ring Speed

Neural Network Implementation and Initialization:

All neural networks are implemented using LSTM architectures with a dedicated initialization strategy. Compared to the previous model version, the application code and training setup were updated due to issues identified in TensorFlow version 2.15 and missing hyperparameter optimization. **The code was therefore updated to TensorFlow 2.18, and the updated conda environment is provided in the project folder (environment.yml).**

Each neural network now consists of two sub-models: a main model (model_inf.keras) and a dedicated initialization model (model_init.keras). The initialization model is only required at the first timestep of a driving cycle and is used to initialize the internal LSTM states of the main model. Typical initialization values are physically reasonable starting conditions, such as a vehicle speed of zero and a battery state of charge of 0.7 for the drivetrain model. During the prediction, the models are loaded only once and reused for all timesteps, which significantly improves application performance.

As a consequence: the inputs of the model_init are scaled with the output scaler, while the inputs to the model_inf are scaled with the input_scaler.

A fully working Python example demonstrating model initialization and inference over complete driving cycles is provided and should be used as reference implementation.

Model Application and Test Cycles:

For the final application, three driving cycles are provided: a standard WLTC cycle and two custom variants referred to as WLTC-low and WLTC-high. The corresponding input data can be found in the *Test_Cycles* folder, while the results generated by the neural networks are stored in the *NN_Application* folder. These cycles were selected during the last project meeting and serve as reference cases for validation and demonstration of the model.